

GAME DEVELOPMENT FOLLOW-UP PROMPT TOOLKIT

PROMPT A: RETHEME OR SPIRITUAL SUCCESSOR

Create a completely new game using the supplied existing project only as a source of design lessons.

Do not modify the reference project.

Do not reuse:

- project title
- executable name
- source namespace
- settings filename
- branding
- lore
- characters
- faction names
- audio mapping
- release notes

Preserve the learned design philosophy:

- responsive native interaction
- data-driven systems
- strong dynamic session variation
- clear progression
- fair AI
- capacity-safe rewards
- atomic upgrades
- readable UI
- high-refresh presentation
- extensive validation
- clean distribution

New configuration

- New genre: [GENRE]
- New theme: [THEME]
- New core fantasy: [FANTASY]
- New roster/content source: [SOURCE]

- New canonical wiki: [WIKI OR DOCUMENTS]
- New visual direction: [STYLE]
- New audio direction: [STYLE]
- New dynamic-run systems: [SYSTEMS]
- New progression: [PROGRESSION]
- New platform: [PLATFORM]

Research the new theme from authoritative sources.

Rebuild all factions, roles, enemies, items, encounters, abilities, and dynamic modifiers around the new canon.

Do not perform a superficial noun swap.

The mechanics themselves should reinforce the new fantasy.

Create a new lore audit documenting verified and inferred assignments.

PROMPT B: MASSIVE ASSET INGESTION

Audit every supplied image, sound, video, localization, and data archive before integration.

Do not manually browse a few filenames and guess.

Create scripts that recursively:

- extract archives
- normalize paths
- record file size
- record format
- decode images
- measure dimensions
- inspect audio duration
- inspect sample rate and channels
- compute hashes
- detect duplicates
- group filename families
- extract numeric IDs
- locate character-linked files
- generate contact sheets
- flag corruption
- identify near-empty files
- identify redundant storefront and promotional content

Create an asset registry with:

- original path
- normalized runtime path
- category
- entity ID
- selected use
- fallback
- required or optional
- duplicate source
- notes

Prioritize integration:

1. Exact character or object match
2. Exact gameplay category
3. Thematically compatible fallback
4. Neutral fallback

Use:

- portraits in compact cards
- full art in drafts, selection scenes, codex entries, and victory screens
- foe art in encounters
- backgrounds in rotating environments
- item art in crafting
- icons in traits and resources
- voice lines only for exact character matches
- BGM through phase-aware pools

Exclude runtime-irrelevant:

- monetization panels
- duplicate package icons
- shop thumbnails
- old tutorial screenshots
- unrelated promotional banners
- redundant quality variants

Document what was integrated and what was intentionally excluded.

PROMPT C: LORE AND FACTION CORRECTION

Audit the complete roster against the designated official wiki and supplied localization.

For every entity, determine:

- canonical name
- ID
- species/type
- faction
- role/class
- origin
- ability
- relationships
- visual motifs
- gameplay adaptation

Identify nonsensical assignments.

Replace invented factions with canonical groups when canonical groups exist.

Do not force every entity into a category solely to equalize counts.

After lore correction, rebalance:

- faction population
- rarity distribution
- role distribution
- early-game access
- late-game access
- synergy breakpoints
- ability identity

Ensure every major faction has viable content across the progression curve.

Document:

- verified assignment
- inferred assignment
- source
- adaptation rationale
- remaining uncertainty

PROMPT D: AI HUMANIZATION

Rebuild opponents so they play the same systems as the player.

Each opponent must possess:

- persistent resources
- persistent shop or decision pool
- normal progression
- storage
- items
- crafting
- legal capacity
- action history
- personality
- difficulty profile

Randomize per session:

- difficulty
- aggression
- economy
- reroll or search frequency
- leveling
- pivot tolerance
- item priority
- scouting
- risk
- preferred strategy

AI must act during planning rather than teleporting into a completed state.

Higher difficulty improves decisions, not hidden stats.

Add tests proving:

- real resource spending
 - real purchases
 - real progression
 - real item use
 - real crafting
 - legal formations
 - strategic divergence
 - no hidden combat multiplier
 - no guaranteed encounter reward
 - no impossible information
-

PROMPT E: PERFORMANCE RESCUE

The game feels sluggish even though it claims a high target frame rate.

Do not solve this by changing only an FPS constant.

Profile and rebuild:

- input-to-frame path
- message processing
- UI animation frequency
- scene reconstruction
- portrait rendering
- scaling
- compositor copies
- simulation frequency
- file access
- resource creation
- effect count

Required changes where applicable:

- immediate presentation after input
- bounded message processing
- high-frequency interactive UI
- event-driven idle UI
- cached portrait atlases
- cached static scenery
- exact-client rendering when possible
- direct presentation for matching sizes
- low-latency scaling during active interaction
- high-quality scaling only for static scenes
- bounded catch-up
- no per-frame asset decoding
- no per-frame graphics-object allocation

Expose telemetry that separates presentation, scene rebuilding, and simulation.

Validate architecture honestly.

Do not claim real monitor performance without a Windows runtime measurement.

PROMPT F: BALANCE RESCUE

A specific encounter, character, item, faction, or strategy feels unbalanced.

Do not immediately nerf or buff the reported object.

Perform a layered audit:

1. Base content power
2. Progression tier
3. Synergy access
4. Item scaling
5. Global modifiers
6. AI behavior
7. Encounter clock
8. Enemy count
9. Enemy defenses
10. Reward value
11. Failure punishment
12. UI clarity

Create deterministic legitimate player states.

Simulate low, average, and high-power bands.

Measure:

- clear rate
- average time
- timeout rate
- survivors
- damage taken
- reward efficiency

Correct the actual cause.

Prefer:

- explicit encounter tiers
- smoother count progression
- longer boss clocks
- capped penalties
- proportional rewards
- restrained capstone stacking

Avoid:

- raw Health multiplication
- arbitrary global nerfs
- removing strategic identity
- balancing around one lucky or unlucky run

Lock the final target bands into automated tests.

PROMPT G: BUGFIX FROM PLAYER REPORT

A player reported:

[PASTE REPORT]

Treat the report as evidence, not a complete diagnosis.

Reproduce the state through:

- source tracing
- deterministic test setup
- UI-state inspection
- capacity inspection
- timeout inspection
- cache inspection
- event-order inspection

Identify whether the issue is:

- mechanical
- visual
- cached state
- capacity
- timing
- input
- documentation
- session modifier
- misunderstanding caused by UI

Fix both:

1. The underlying issue
2. The communication failure that made it confusing

Add a regression test reproducing the exact reported sequence.

Audit adjacent systems for the same pattern.

Do not merely add a special-case patch around the reported object.

PROMPT H: DISTRIBUTION RELEASE

Prepare the project for strangers.

Assume the player:

- extracts it incorrectly
- deletes an optional file
- deletes a mandatory file
- has multiple monitors
- changes DPI
- runs unusual resolutions
- presses Escape accidentally
- relaunches later
- checks Windows file properties
- verifies hashes
- inspects the ZIP
- builds the source
- runs antivirus
- has no development tools installed

Add or verify:

- clear folder structure
- executable icon
- version metadata
- portable settings
- settings reset instructions
- missing-file diagnostics
- optional fallbacks
- quit confirmation
- off-screen-window recovery
- player README
- controls
- troubleshooting
- build guide
- checksums
- verification scripts
- deterministic build
- clean source package

Run clean-room extraction and rebuild.

The rebuilt executable must match the shipped executable when deterministic linking is available.

PROMPT I: CONTENT EXPANSION WITHOUT BREAKING THE CORE

Add:

[NEW CHARACTERS / ENEMIES / MAPS / ITEMS / MUSIC / BOSSES / MODES]

Requirements:

- preserve save compatibility when applicable
- preserve existing data IDs
- do not renumber content casually
- update registries and documentation
- use exact lore research
- maintain rarity and faction distribution
- update AI evaluation
- update asset mapping
- update audio mapping
- update balance simulations
- add regression tests
- avoid increasing load time without purpose
- avoid loading all new assets into memory
- keep optional content optional
- version the release
- provide a changelog

Before packaging, compare:

- old performance
- new performance
- old package size
- new package size
- old balance bands
- new balance bands

PROMPT J: MODULAR GENRE CONVERSION

Convert the current design philosophy into a new genre:

- Existing genre: [OLD GENRE]

- New genre: [NEW GENRE]
- Theme: [THEME]
- Core fantasy: [FANTASY]

Preserve abstract strengths:

- responsiveness
- dynamic sessions
- strategic build expression
- fair AI
- clear upgrades
- capacity-safe rewards
- strong effects
- phase-aware audio
- high-quality distribution

Do not preserve mechanics that no longer fit.

Translate systems by function.

Examples:

- autobattler roster → party, squad, deck, garage, or army
- shop rerolls → loot choices, mission board, drafting, recruitment
- fusion → crafting, evolution, promotion, weapon upgrading
- traits → perks, tags, doctrines, schools, set bonuses
- Carousel → shared draft, loot room, salvage phase, auction
- patron blessing → sponsor, deity, commander, mutation, faction contract
- PvE Incursion → boss wave, raid, anomaly, siege, elite mission
- bench overflow → reserve, stash, garage, convoy, temporary deployment

Rebuild UI, pacing, balance, and lore around the new genre rather than preserving old terminology.

FINAL TOOLKIT RULE

For every follow-up task:

- use the latest complete source
- preserve working systems
- identify the real cause
- fix adjacent failure paths
- add regression coverage
- update documentation
- rerun isolated validation gates
- package from frozen source
- clean-room rebuild

- report honest limitations